A Numerical Approach to Trigonometry

Implementing Taylor Series in Programming

Jaehoon Song

March 26, 2026

Introduction

- The intersection of Calculus and Software Engineering is explored.
- A fundamental question is addressed: How are irrational values calculated by hardware?
- The transition from infinite mathematical theory to finite computer logic is demonstrated.

The Computational Problem

Hardware Limitations

Digital circuits are inherently limited to basic arithmetic operations:

- Addition and Subtraction
- Multiplication and Division

Continuous functions, such as $\sin(x)$, cannot be stored as an infinite lookup table. Instead, these values must be approximated using iterative algorithms.

The Taylor Series Expansion

The $\sin(x)$ function can be represented as an infinite sum of powers:

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!}$$

This expansion is simplified into a sequence of polynomial terms:

$$\sin(x) \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Higher accuracy is achieved as more terms are calculated.

Implementation in Java

The mathematical summation is translated into a `for` loop.

Java Logic

```
double x = 1.0;
double result = 0;

for (int i = 0; i < 5; i++) {
  double term = Math.pow(x, 2*i + 1) / factorial(2*i + 1);
  if (i % 2 == 0) {
    result += term;
  } else {
    result -= term;
  }
}
```

Validation of Results

The approximation is compared against the standard library:

- **Taylor Approximation (5 terms):** 0.84147101...
- **Java Math.sin(1):** 0.84147098...

Conclusion

Mathematical abstraction is bridged by software loops. This logic serves as the foundation for modern computational tools.

A Numerical Approach to Trigonometry

Implementing Taylor Series in Programming

Jaehoon Song

March 26, 2026

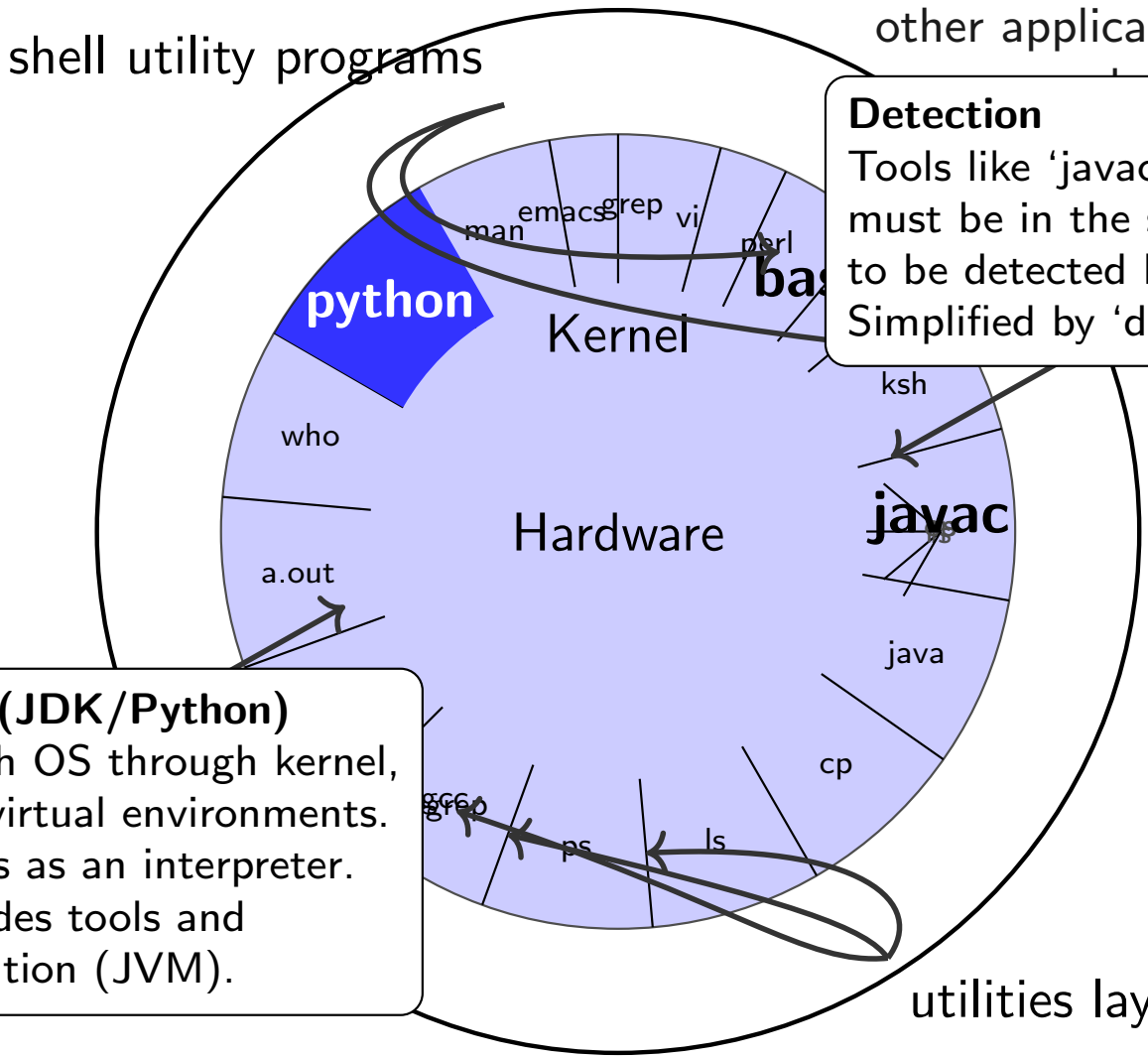
Abstraction: Simplifying Computation

- **Definition:** Hiding complex details behind a simpler interface.
- **OS Abstraction:** Managing hardware resources and providing system calls.
- **Shell Abstraction:** A command-line interface to interact with the OS and execute utilities.
- **Runtime Abstraction (JDK/Python):** Running code on a virtual machine (JVM) or interpreter, abstracting away underlying system differences.
- **Environment Abstraction:** Automating setup with tools like ‘dev.bat‘ / ‘dev.sh‘ to quickly find and detect these layers.

Fast Path

We can quickly overview the relationship of the OS, Shell, JDK, and Python by visualizing their place in the system hierarchy.

System Hierarchy (Modified image_0.png)



Detection
Tools like ‘javac‘ and ‘python‘ must be in the system PATH to be detected by the shell. Simplified by ‘dev.sh‘ automation.

Runtimes (JDK/Python)
Iterate with OS through kernel, providing virtual environments. Python sits as an interpreter. JDK provides tools and Java execution (JVM).

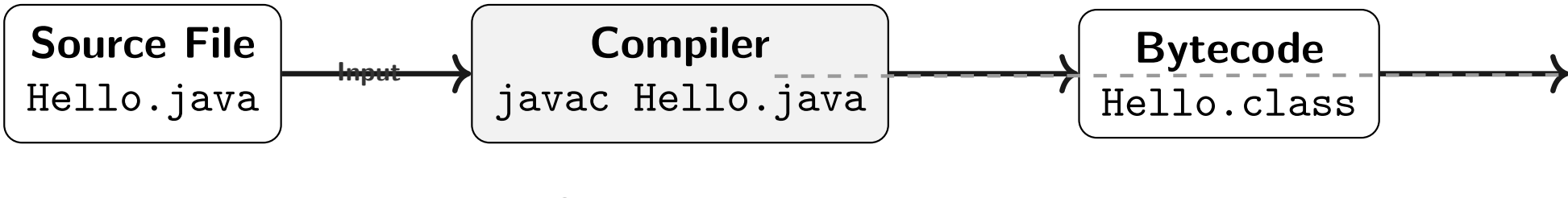
Environment Setup & Detection

- **Finding the Tools:** When a command is typed (‘javac Hello.java‘), the shell searches the system ‘PATH‘ environment variable.
- **PATH Variable:** A semicolon- or colon-separated list of directories containing executables.
- **Example Detection Sequence:**
  1. User types command.
  2. Shell looks in directories defined in ‘PATH‘.
  3. First executable found is run (or error).
- **Automation via `dev.sh` / `dev.bat`:** Conceptually, these scripts modify the local session environment to automatically detect these tools.

Conceptual `dev.sh` Snippet

```
# Define base paths (detected or predefined)
export JDK_HOME="/path/to/jdk-17"
export PYTHON_HOME="/path/to/python3.9"
```

Java: Compile-Once, Run-Everywhere



Validation

The code we write is first compiled into a platform-independent format and then run by the JVM, abstracting away system differences.

Python: Interpreted & Dynamic

Reads script
Executes immediately

Validation

Python code is interpreted and executed line-by-line, leading to a much faster write-and-run cycle than the compiled Java approach. Both have a place within the system.